

Swapnil Roy

In Kee Kim

CSCI 4730

24 October 2025

README.pdf

My Work:

```
1: #include <stdio.h>
2: #include <stdlib.h>
3: #include <arpa/inet.h>
4: #include <pthread.h>
5: #include "webserver.h"
6:
7: #define MAX_REQUEST 100
8:
9: int port, numThread;
10:
11: void *listener()
12: {
13:     int r;
14:     struct sockaddr_in sin;
15:     struct sockaddr_in peer;
16:     int peer_len = sizeof(peer);
17:     int sock;
18:
19:     sock = socket(AF_INET, SOCK_STREAM, 0);
20:     sin.sin_family = AF_INET;
21:     sin.sin_addr.s_addr = INADDR_ANY;
22:     sin.sin_port = htons(port);
23:     r = bind(sock, (struct sockaddr *) &sin, sizeof(sin));
24:     if(r < 0) {
25:         perror("Error binding socket:");
26:         return;
27:     }
28:
29:     r = listen(sock, 5);
30:     if(r < 0) {
31:         perror("Error listening socket:");
32:         return;
33:     }
34:
35:     printf("HTTP server listening on port %d\n", port);
36:     while (1)
37:     {
38:         int s;
39:         s = accept(sock, NULL, NULL);
40:         if (s < 0) break;
41:
42:         process(s) producer(s)
43:     }
44:     close(sock);
45: }
46:
2.18
```

The hint from the instructor notes, said to change the ‘**process()**’ from the single threaded webserver version, into a **producer** for the multi-threaded listener() function. This is how I did that. My code is below:

```

/* Producer Loop */
while (1)
{
    /* Accept Incoming Connections */
    int s = accept(sock, NULL, NULL); // accept incoming connection
    if (s < 0) { // if accept fails
        perror("accept failed"); // print error message
        break; // exit loop
    } // if

    producer(s); // enqueue the client socket fd into the shared buffer
} // while

close(sock); // close the server socket
pthread_exit(NULL); // exit thread

```

This is within the **listener** function. In the single-threaded version, the **listener()** directly calls **process()** to handle each client connection sequentially. In the multi-threaded version, I replaced **process()** with a **producer()** call, meaning that the listener thread now acts as the producer in a producer-consumer model, as it just accepts sockets and enqueues them into a shared bounded buffer (`requestBuffer[]`). This is the main modification I made to the listener call code, whilst the other methods were fully new by me.

```

void thread_control() {
    // Initialize Synchronization Parameters
    pthread_mutex_init(&mutex, NULL); // initialize mutex for buffer access
    sem_init(&sem_empty, 0, MAX_REQUEST); // initialize empty semaphore to MAX_REQUEST
    sem_init(&sem_full, 0, 0); // initialize full semaphore to 0

    // Error Handling
    if (numThread > MAX_REQUEST) { // if number of threads exceeds MAX_REQUEST
        numThread = MAX_REQUEST; // cap number of threads at MAX_REQUEST
    } // if
    if (pthread_create(&listenerThread, NULL, listener, NULL) != 0) { // if thread creation fails
        perror("pthread_create(listener)"); // print error message
        exit(1); // exit program
    } // if
    // create worker threads (consumers)
    for (int i = 0; i < numThread; i++) { // for each worker thread
        if (pthread_create(&workerThreads[i], NULL, worker, NULL) != 0) { // if thread creation fails
            perror("pthread_create(worker)"); // print error message
            exit(1); // exit program
        } // if
    } // for

    /* Thread Join*/
    pthread_join(listenerThread, NULL); // join the listener thread
    for (int i = 0; i < numThread; i++) { // if listener ever exits,
        pthread_join(workerThreads[i], NULL); // join worker threads
    } // for

    /* Cleanup - Not Usually Reached */
    pthread_mutex_destroy(&mutex); // destroy mutex
    sem_destroy(&sem_empty); // destroy empty semaphore
    sem_destroy(&sem_full); // destroy full semaphore
}

```

Thread control was also fully implemented, joining all the threads at the end so that there are no orphaned children. I also error checked to make sure that we were not going over the max thread count. The worker(s) were called from here to make sure there were no errors. All the synchronization mutex and such was destroyed at the end.

Testing Webserver (Single Threaded):

10 Requests:

```

[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 10
Request: GET 127.0.0.1:5001/, # of client: 10

```

```
Time to handle 10 requests (0 failed): 10.005617 sec

real    0m10.010s
user    0m0.005s
sys     0m0.006s
```

Testing Multi-Threaded Server (10 Threads):

```
[sr42946@csci-odin proj2]$ ./webserver_multi 5001 10
HTTP server listening on port 5001
```

- **Client Time Code:** measures how long it took for the threads to finish their HTTP
- **Shell Time:** measures the entire process runtime from the OS perspective.

10 Requests:

```
[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 10
Time to handle 10 requests (0 failed): 1.008333 sec

real    0m1.013s
user    0m0.003s
sys     0m0.007s
```

20 Requests:

```
[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 20
Time to handle 20 requests (0 failed): 2.053719 sec

real    0m2.058s
user    0m0.002s
sys     0m0.011s
```

70 Requests:

```
[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 70
Time to handle 70 requests (0 failed): 7.008540 sec

real    0m7.013s
user    0m0.005s
sys     0m0.023s
```

Testing Multi-Threaded Server (50 Threads):

Server Side For 50 Threads

```
[sr42946@csci-odin proj2]$ ./webserver_multi 5001 50  
HTTP server listening on port 5001
```

10 Requests:

```
[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 10  
Request: GET 127.0.0.1:5001//, # of client: 10
```

```
Time to handle 10 requests (0 failed): 2.029351 sec
```

```
real    0m2.034s  
user    0m0.006s  
sys     0m0.005s
```

20 Requests:

```
[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 20  
Request: GET 127.0.0.1:5001//, # of client: 20
```

```
Time to handle 20 requests (0 failed): 1.204963 sec
```

```
real    0m1.210s  
user    0m0.004s  
sys     0m0.011s
```

70 Requests:

```
[sr42946@csci-odin proj2]$ time ./client 127.0.0.1 5001 70  
Request: GET 127.0.0.1:5001//, # of client: 70
```

```
Time to handle 70 requests (0 failed): 2.015390 sec
```

```
real    0m2.021s  
user    0m0.008s  
sys     0m0.022s
```

Testing Results:

With a smaller number of threads, the webserver shows degradation. More specifically, there was longer end-to-end request processing time. For example, the single-threaded webserver code in processing 10 clients took around **10 seconds**. With only one worker thread, the server handles client requests sequentially, one request must finish before the next begins.. Since each process() call simulates a 1-second workload, 10 requests take roughly 10 seconds to complete.

With a larger number of threads, the webserver shows performance improvement. More specifically, there was shorter end-to-end request processing time. For instance, as the number of worker threads increases, the server can process multiple client connections concurrently, reducing the total elapsed time per batch of requests. In the pictures, it shows that with 10 threads, 10 concurrent clients can finish in about **1 second**. Even with larger workloads (20 and 70 requests), the total time **scales roughly by 1 second for each 10 requests**.

When the 50 threaded model was induced, the scaling reduced drastically, going from 70 clients taking 7 seconds, to 70 clients only taking 2 seconds for the 50 threaded web server. This proves that even with just 100 threads, we can achieve close to **real-time** results from our archaic webserver, indicating the power of threads, concurrency, and simultaneous processing, that **reduces total end-to-end time**.

Shared Data Structure and Synchronization Design:

- I correctly use the learnings from class
 - Semaphore
 - Mutex lock
 - Consumer-producer model
- For example, this is what I based it off from

Producer and Consumer

```
while (true) { Producer (w/ Semaphore)
    . . .
    /* produce an item in next_produced */
    . . .
    wait(empty);
    wait(mutex);
    . . .
    /* add next_produced to the buffer */
    . . .
    signal(mutex);
    signal(full);
}

while (true) { Consumer (w/ Semaphore)
    wait(full);
    wait(mutex);
    . . .
    /* remove an item from buffer to next_consumed */
    . . .
    signal(mutex);
    signal(empty);
    . . .
    /* consume the item in next_consumed */
    . . .
}
```

I implemented the wait() and signal() with the mutex and semaphores perfectly for both the producer and the consumer.

Producer:

```
void producer(int client) {
    /* Wait For Empty Slot */
    sem_wait(&sem_empty); // wait for an empty slot in the buffer
    pthread_mutex_lock(&mutex); // lock the mutex to protect buffer access

    /* Add Client To Buffer*/
    requestBuffer[produceIndex] = client; // add client fd to the buffer at produceIndex
    produceIndex = (produceIndex + 1) % MAX_REQUEST; // update produceIndex in a circular manner

    /* Signal Full Buffer */
    pthread_mutex_unlock(&mutex); // unlock the mutex
    sem_post(&sem_full); // signal that there is a new full slot in the buffer
} // producer
```

Within the **semaphore.h** import in the C language, **sem_post()** translates to **signal()**. Just like the image above, for the producer, I did **wait(empty)**, **wait(mutex)**, **signal(mutex)**, **signal(full)** in the order as prescribed by the image. We want to produce when the bounded buffer is empty, and not produce when the bounded buffer is full.

Descriptions of each:

- **sem_wait(&sem_empty):** locks if the buffer is full (no empty slots).
- **pthread_mutex_lock(&mutex):** exclusive access to requestBuffer. We write the socket into requestBuffer[produceIndex]. We advance produceIndex circularly.
- **pthread_mutex_unlock(&mutex):** let other threads touch the buffer
- **sem_post(&sem_full):** announce that there is another request ready to be made

Consumer:

```
void *worker(void *arg) {
    (void)arg; // suppress unused parameter warning

    while (1) { // infinite loop
        /* Wait For Full Buffer */
        sem_wait(&sem_full); // do not consume from an empty buffer
        pthread_mutex_lock(&mutex); // lock buffer, remove one fd
        int client = consumer(); // get client fd from buffer
        pthread_mutex_unlock(&mutex); // unlock buffer
        sem_post(&sem_empty); // signal that there is a new empty slot in the buffer

        /* Process Client Request */
        process(client); // from net.c: process the client request
        close(client); // close the client socket
    }
    return NULL; // needs to return a value due to pthreads
}
```

Just like the image above, for the producer, I did **wait(full)**, **wait(mutex)**, **signal(mutex)**, **signal(empty)** in the order as prescribed by the image. We want to consume when the bounded buffer is full, and not consume when the bounded buffer is empty.

Descriptions:

- **sem_wait(&sem_full);** if there are no ready requests, then block
- **pthread_mutex_lock(&mutex):** We lock the mutex, remove one fd from requestBuffer[consumeIndex]
- **pthread_mutex_unlock(&mutex):** then we unlock the mutex.
- **sem_post(&sem_empty):** Announces that there's now one more free slot in the buffer, inviting the worker thread to actually handle the HTTP request via **process(client)**.

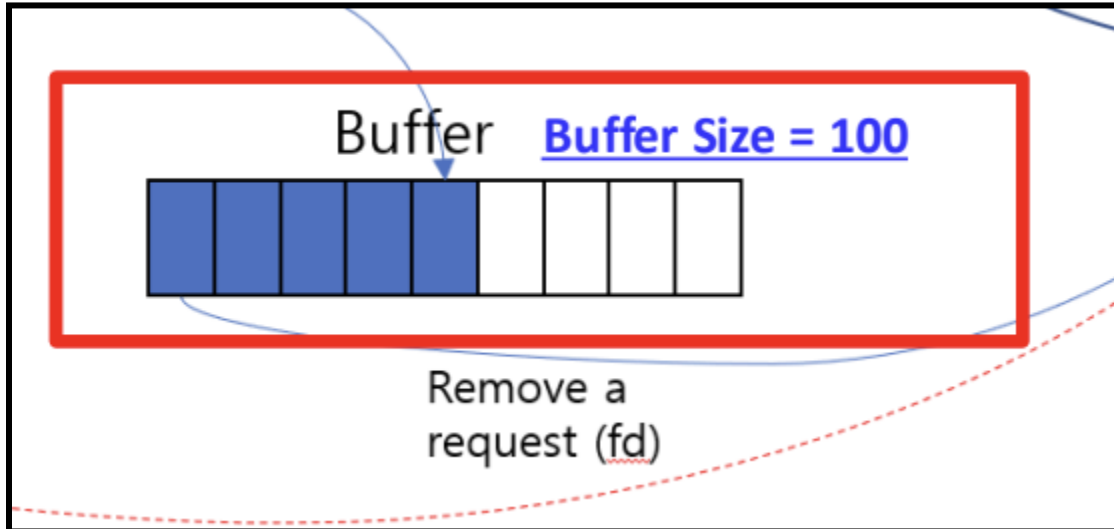
Bounded Buffer Data Structure:

In class, we said there is a shared buffer that both producer and consumer access. Access to that buffer must be synchronized so producers and consumers don't step on each other.

```

/* Shared Bounded Buffer For Incoming Requests */
int requestBuffer[MAX_REQUEST]; // shared buffer for incoming requests
int produceIndex = 0; // next write index in the buffer
int consumeIndex = 0; // next read index in the buffer

```



I implemented that ‘buffer box’ above. **requestBuffer** is the bounded buffer that holds incoming client sockets. **produceIndex** and **consumeIndex** implement a circular queue. The **BUFFER_SIZE** is set to 100 to indicate that it is a buffer of size 100 that can only handle 100 requests at most. This **BUFFER_SIZE** is the catalyst for the bounded buffer problem that is solved by our synchronization strategies.

No Concurrency Problems in Design:

My output shows that I had 0 clients fail on me on all my runs.

```

Time to handle 10 requests (0 failed): 2.029351 sec

```

No Deadlocks:

Deadlock: happens if two or more threads get stuck waiting on each other forever, mostly because they acquire locks in inconsistent orders, or they wait on semaphores in an order that

My design does not deadlock because both the producer (listener thread) and the consumers (worker threads) always acquire locks and semaphores in the same fixed order:

- Producer:
 - `sem_wait(&sem_empty);`
 - `pthread_mutex_lock(&mutex);`

- write to buffer
- pthread_mutex_unlock(&mutex);
- sem_post(&sem_full);
- Consumer:
 - sem_wait(&sem_full);
 - pthread_mutex_lock(&mutex);
 - read from buffer
 - pthread_mutex_unlock(&mutex);
 - sem_post(&sem_empty);

This matches the standard bounded-buffer solution from class, where all threads respect the same lock discipline, where the mutex is only ever held while touching the buffer, and it is always released before signaling the opposite semaphore. That prevents circular wait that caused deadlock in the too much milk problem within attempt number 2.

Runtime Evidence:

- I hammer the server with 10, 20, 70 concurrent client connections within multiple worker configurations, and in every case, the server keeps accepting requests and finishing them
- We never saw the system hang permanently, and every run printed:

No Race Conditions:

Race Conditions (Multiple Threads): When two threads access a critical section and cause undecided output. May be caused by

- read/write requestBuffer, produceIndex, or consumeIndex (critical sections) at the same time without proper locking,
- If two workers pulled the same client socket fd, or corrupted produce/consume Index

How I Solve That:

- All accesses to the shared circular buffer (requestBuffer) and its indices (produceIndex, consumeIndex) are wrapped by the same mutex
- Semaphores (sem_full / sem_empty) guarantee that consumers never remove from an empty buffer and producers never write to a full buffer

Runtime Evidence

- In stress tests, every request completed and 0 failed was reported, even at 70 connections
- Thus, no data corruption, duplicate consumption, or crashes we observed, meaning that no race conditions were presented in.
- All the output looked like where it should be, thus no race conditions